# ELEC 278 – Data Structures and Algorithms

## Lab 4 : Sorting Algorithms

November 1 - November 11

## Contents


1. Basic Lab Information ..... 2

1.1 New Instructions for this lab ..... 2

1.2 Overview and Background ..... 2

2. Lab Resources ..... 2

2.1 List of Files provided with this Lab ..... 3

3. Lab Objective ..... 3

3.1 Topics covered in this lab ..... 3

4. Lab Steps ..... 3

4.1 Overview ..... 3

4.2 Step 1 – Deliverable 1 ..... 4

4.3 Step 2 - Deliverable 2 ..... 5

4.4 Step 3 - Deliverable 3 ..... 6

4.5 Summary of Deliverables ..... 7



Copyright 2021 David Athersych. This material is provided for use of students registered in ELEC278 at Queen's University, Kingston in the Fall semester 2020.


**Document History**

| Date | Author | Description |
|---|---|---|
| 2020-10-25 | DFA | Complete rewrite |
| | | |
| | | |

# 1. Basic Lab Information

You should review the Lab Overview document prior to reading the material for this lab exercise.

If you have already installed a C compiler on your computer (or have one available), then you can continue with the lab. If you have not installed the C programming tools, please stop working on this lab, and review Lab Overview document Appendix A. You may also wish to review Tutorial 1 before you attempt this lab.

Be sure to note the instructions for submitting your work. Note that it is acceptable that you submit more than one version of your work. ***In the absence of any indication to the contrary***, the markers will only look at your latest submission.

You may also find it useful to review previous labs as you work on this lab exercise.

## 1.1 New Instructions for this lab

As announced on onQ, you will be permitted to work with one partner on Lab 4. It is not necessary to work with a partner, but you may find it more instructive to work with someone to discuss ideas with. Both partners are expected to contribute equally to the work handed in and both partners will be awarded the same grade for the work.

The work handed in by two-person lab team **MUST** include the names of both partners. **BOTH PARTNERS WILL SUBMIT A COPY OF THE LAB WORK.** The markers will review one of the copies and will award both team members the same mark.

Carefully note any changes to the instructions for submitting lab work, found in Section 4.

## 1.2 Overview and Background

This lab exercise is designed to help learners become more familiar with the sorting algorithms discussed in week 8 and 9 of ELEC278.

# 2. Lab Resources

Specific to this lab:

1. Lab Instructions (this document)
2. A zip file containing source code (called Lab04Materials.zip). Check the file “PACKING.LIST” (it is a plain text file) for a list of files that are included.

Learner provided:

1. Programming environment selected by the learner. The Lab Overview document provides a list

of possible C programming tools that you may use.

This lab has code for you to read. The purpose of the example code is to reinforce what you will see in the class material. It also provides you with code to cut-and-paste into the code you write to solve the assigned problems. This lab has multiple programming problems to solve.

### 2.1 List of Files provided with this Lab

(See the file PACKING.LIST included in the Lab04Materials.zip file for the most up-to-date list.)

## 3. Lab Objective

There are three objectives for this lab:

1) Review and test supplied code for all the sort routines discussed in class.
2) Compare sort performance for all sort routines, using several large sets of data.
3) Change the Quicksort code to use Insertion Sort when the partition size gets below a certain size.

### 3.1 Topics covered in this lab

- Sorting algorithms.
- Performance comparison for various sorts.

## 4. Lab Steps

### 4.1 Overview

This lab includes three steps, all of which involve using the code provided to you in the zipped directory **lab04Materials.zip.**

Before you get down to the deliverables, you will examine all the source files that start with **L4_**. There are files for Bucket Sort, Bubble Sort, Heap Sort, Insertion Sort, Merge Sort, Quicksort, Radix Sort, Selection Sort, and Shell Sort. There is also a file called L4_TESTSORT.C which you will use as the basis for your code, and another file called L4_UTILITYCODE.C that contains a number of functions useful for all the sorts – such as a function to swap data at two locations in an array (and keep track of how many swaps are done), a function to print the contents of an array, and a function that will confirm that your array is sorted – either in ascending or descending order.

You will note that L4_TESTSORT **`#include`**s all the other source files. Those with more experience using development environments may choose to separately compile each source file and then link all the object files together, but the technique suggested just makes it a little easier to get your code working. Note that all source files have lots of comments to help explain what the code is doing and how to use the code.

You will also note a file called MAKEDATA.C. This is a program to create the three data files

RANDOM.TXT, ASCENDING.TXT and DESCENDING.TXT. You can see how this program writes data to files and you can compare the writing of data to the code in L4_UTILITYCODE.C that reads the files.

MAKEDATA will generate 3 files all with an equal number of data items. It is run as follows:

```
makedata
```

or

```
makedata  42000
```

In the first case, three files are created containing a default number of items (100000). In the second case, the program is run with a command line parameter – 42000. The parameter specifies how many data items each file should contain.

You can use the program to create small datasets for initial testing, and then for timing purposes, you may wish to create a much bigger dataset. (There are a couple of small RANDOM files already provided.)

**Note regarding submitted file specifications:** The specifications for the files you are to submit include a specification for how to include your name – LastName, FirstName. If you are working with a partner, then that requirement is modified to be:

LastName1, FirstName1
LastName2, FirstName2

It would be helpful if you sort the list of partners so that LastName1 is alphabetically smaller than LastName2.

### 4.2 Step 1 – Deliverable 1

Step 1 is basically a familiarization step. You will find code in main() that looks like:

```
swaps = 0;
start = clock();
//rslt = bubblesort (data, 0, MAXINDEX);
//rslt = insertionsort (data, MAXINDEX+1);
//rslt = mergesort (data, 0, MAXINDEX);
rslt = quicksort (data, 0, MAXINDEX);
//rslt = selsortI (data, 0, MAXINDEX);
//rslt = selsortR (data, 0, MAXINDEX);
//rslt = ShellSort (data, MAXINDEX+1);
end = clock();
printarray ("Final array:", data, 0, MAXINDEX, 10);
```

This code has lines commented out, and one line in the middle of the commented-out lines that calls

a sort routine – in this case Quicksort. Step 1 is to use this code to test the provided sort routines, using the data provided or data of your own. You will need to change the value of the global variable **`verbose`** depending on whether you want to see lots or little output. It is suggested that you use a small set of data (20-40 data items) and look at all the verbose output – especially for quicksort, mergesort and heapsort. Follow what gets displayed and make sure you understand how each sort works.

Also note the behavior when the input is already sorted, and when the input is sorted in the reverse order. Note which algorithms detect the sorted case quickly and avoid unnecessary work.

Using an input file with 30 to 40 data items in random order, capture the **output from Quicksort** in a file called LAB5_PART1.TXT. Make sure you have edited this file to include your LastName, FirstName at the top of your file. Make sure you understand the progression of the algorithm through each partition step.

### 4.3 Step 2 - Deliverable 2

For Part 2, you will use three data files that have been supplied – RANDOM.TXT, ASCENDING.TXT and DESCENDING.TXT. These files contain data in an order suggested by their titles – RANDOM contains random data, ASCENDING contains a set of data already sorted in ascending order and DESCENDING contains data already sorted in descending order.

Note that the sort routines supplied all sort in ascending order.

You will modify the L4_TESTSORT code so that it performs **every** sort algorithm on **each** of these three files. Note that the code shown above in Section 4.2 has two calls to a routine called clock() – these routines capture an internal clock time. Do not be too concerned about the units – all you care about is the total sort time (end-start) for each sort that is performed. Keep all the data in a table – each sort will have all its data in one row, with the columns being the three files that you run the sort on.

Output the data as a readable table in a text file. The following table provides an example for what your output should look like. Your table does not need the lines. Your table will have all the sorts listed, for all the data orderings. The numbers in this table have been made up for illustrative purposes only.

| | | | | | | |
|---|---|---|---|---|---|---|
| Lastname, | FirstName | Dataset size: | 100000 | | | |
| | RANDOM | | ASCENDING | | DESCENDING | |
| Sort | Elapsed Time | Number of Swaps | Elapsed Time | Number of Swaps | Elapsed Time | Number of Swaps |
| Bubble Sort | 800 | 1000000 | 800 | 0 | 800 | 1000000 |
| Merge Sort | 700 | 500000 | 600 | 0 | 800 | 1000000 |
| Quicksort | 450 | 350000 | 350 | 40 | 800 | 1000000 |
| Etc. | | | | | | |

Call your file LAB5_PART2.TXT. Make sure you have included in this file your LastName, FirstName at the top of your file.

***Be careful of a pitfall here.*** If you read the data from a file into an array, then sort the array, you cannot use that array again for the second sort – it will not be the same starting point. So, you may have to either make a copy of the array you read in or reread the file prior to a subsequent sort. If you reread the file, do not forget to free() the prior array. Also, do not include the reading of the data in your time calculation for actual sorting.

### 4.4 Step 3 - Deliverable 3

In the third step, you will review the Quicksort and the Insertion Sort code. You will recall from the class presentation that it was suggested that Quicksort is not very efficient when the partition sizes become very small. It was also suggested in class that most implementations of Quicksort switch to another method when the partition size gets small.

Create a new version of the Quicksort code to switch to Insertion sort on partitions less than 32 data items in size.

1. Confirm, using a small data set, that your version works.
2. Run a comparison between the original Quicksort and your new version in a way like the way you did comparisons in Step 2. Use the RANDOM.TXT file for the data to test both versions – with at least 50000 items of data in your input file.

For this part, you will submit two files:
L4_QUICKSORT2.C – your version of updated Quicksort,
LAB4_PART3.TXT – the output of your comparison between the two versions of Quicksort. The format of this file shall be as follows (the lines around the data items are not required):

| | | |
|---|---|---|
| LastName,FirstName | Size of dataset | 100000 |
| | Elapsed time | Number of swaps |
| Original Quicksort | 99999999 | 5555555 |
| Modified Quicksort | 88888888 | 4444444 |

### 4.5 Summary of Deliverables

LAB4_PART1.TXT
LAB4_PART2.TXT
L4_QUICKSORT2.C
LAB4_PART3.TXT

### 4.6 Submission Instruction

1- Make sure you write your name and your partner name inside the submitted files.
2- Each students submit a copy of her/his group separately for marking.
3- Place all your source codes and necessary files in a folder named in the following format:
   278-1234-Lab4 where 1234 stands for the last 4 digits of your student ID,
   e.g.: For student with ID 20196072, the folder should be named 278-6072-Lab4.
4- Compress the above folder using Zip (the extension must be .zip).
5- Upload this zipped file into the drop box of Lab4.